CP020003

ARTIFICIAL INTELLIGENCE

Week 09

Time Series Forecasting

From Classical Statistics to Transformers

A complete field guide — built and taught on real NASDAQ Composite (IXIC) daily data.

Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

Khon Kaen University · 2026

!

A Quick, Honest Disclaimer

This lecture teaches **forecasting techniques** using stock-index data — because financial time series are freely available, richly documented, and genuinely hard to predict.

It is NOT investment advice, and it is definitely not a signal to go all-in on NASDAQ tomorrow morning.

(In fact — by the end of this deck — you'll understand exactly why beating the market is so hard. That's kind of the whole point.)

SECTION 1 · THE BIG PICTURE

Why Time Series Is Everywhere

Any time you measure something repeatedly and the order matters, you have a time series — and someone, somewhere, is trying to forecast it.

Finance

Stock indices, FX rates, trading volume — today's lecture

Weather & Climate

Temperature, rainfall, storm tracks, 10-day forecasts

Supply Chain

Demand forecasting, inventory, delivery ETAs

Energy

Power-grid load, renewable output, smart meters

Healthcare

ECG signals, patient vitals, epidemic curves

Tech Ops

Server CPU load, API traffic, anomaly detection

What Is a Time Series?

Time series: a sequence of values measured at successive, ordered points in time — x

1, x2, x3, … , xt. Shuffle the order, and the series stops meaning anything.

Ordinary Tabular ML vs. Time Series

Tabular ML	Time Series
Rows are (usually) independent	Each point depends on the ones before it
Shuffling the dataset is fine	Shuffling destroys the signal
Random train/test split	Must split chronologically — no peeking at the future
One snapshot in time	Trend, seasonality & regime shifts evolve over time

SECTION 2 · THE DATA

Meet the Dataset: Global Stock Indices

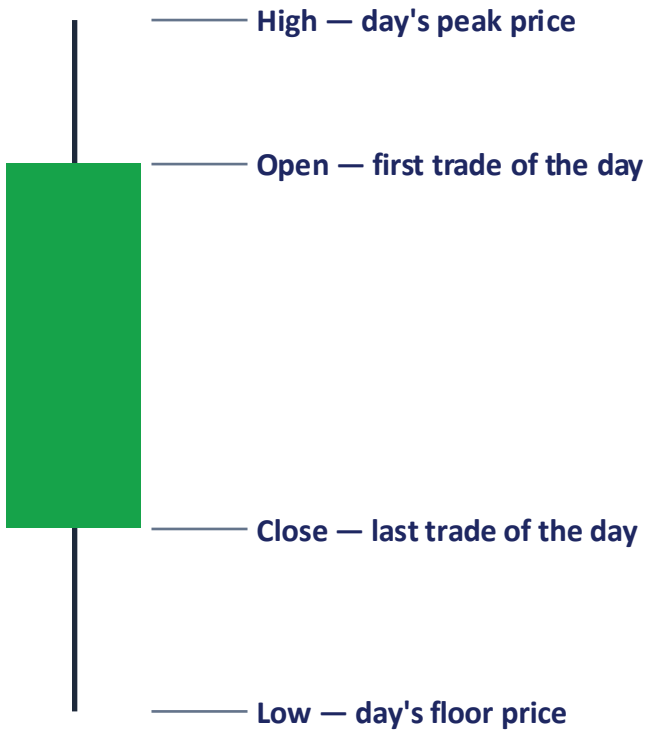
Daily data for 13 major stock indices, pulled straight from the course repository.

indexData.csv	Raw daily OHLCV per index — the unprocessed source file
indexInfo.csv	Metadata: which index belongs to which country, exchange & currency
indexProcessed.csv	Cleaned OHLCV + a currency-normalized CloseUSD column — what we use today

We teach on IXIC — the NASDAQ Composite — using ~6.5 years of daily trading data (Jan 2015 → May 2021, 1,613 trading days).

Other indices in the file: NYA, HSI, 000001.SS, N225, N100, 399001.SZ, GSPTSE, NSEI, GDAXI, KS11, SSMI, TWII, J203.JO — Wall Street to Shanghai to Johannesburg.

Decoding OHLCV — Anatomy of a Trading Day



Open First traded price when the market opens that day

High Highest traded price reached during the day

Low Lowest traded price reached during the day

Close Last traded price when the market closes — the number everyone quotes, and what we forecast

Adj Close Close price adjusted for dividends/splits — a fairer long-run comparison

Volume Total number of shares traded that day — a proxy for interest & liquidity

CloseUSD Close price converted to USD, so every index sits on the same currency scale

Why We Forecast Close (and Why CloseUSD)

① Close is the market's verdict

Open, High and Low describe what happened during the day. Close is the price everyone agreed on by the end — it's what's quoted in the news, used to compute daily returns, and the standard target for forecasting.

② Volume confirms conviction

A price move on huge volume means many traders agreed; on thin volume, it can just be noise. We'll use it as a feature later.

③ CloseUSD levels the playing field

13 indices, 13 local currencies — Yen, Yuan, Rand, Euro, Rupee... Comparing raw Close values across countries is meaningless. CloseUSD converts every index to US dollars so magnitudes are comparable and models can be reused across markets.

Today's lecture forecasts IXIC's Close directly — it's already USD-denominated, so Close and CloseUSD are the same series.

Always Look Before You Model

Plots reveal trend, volatility regimes and outliers that summary statistics hide. This is real IXIC data.

Min close

$4,266

Max close

$14,175

Mean close

$8,563

Trading days

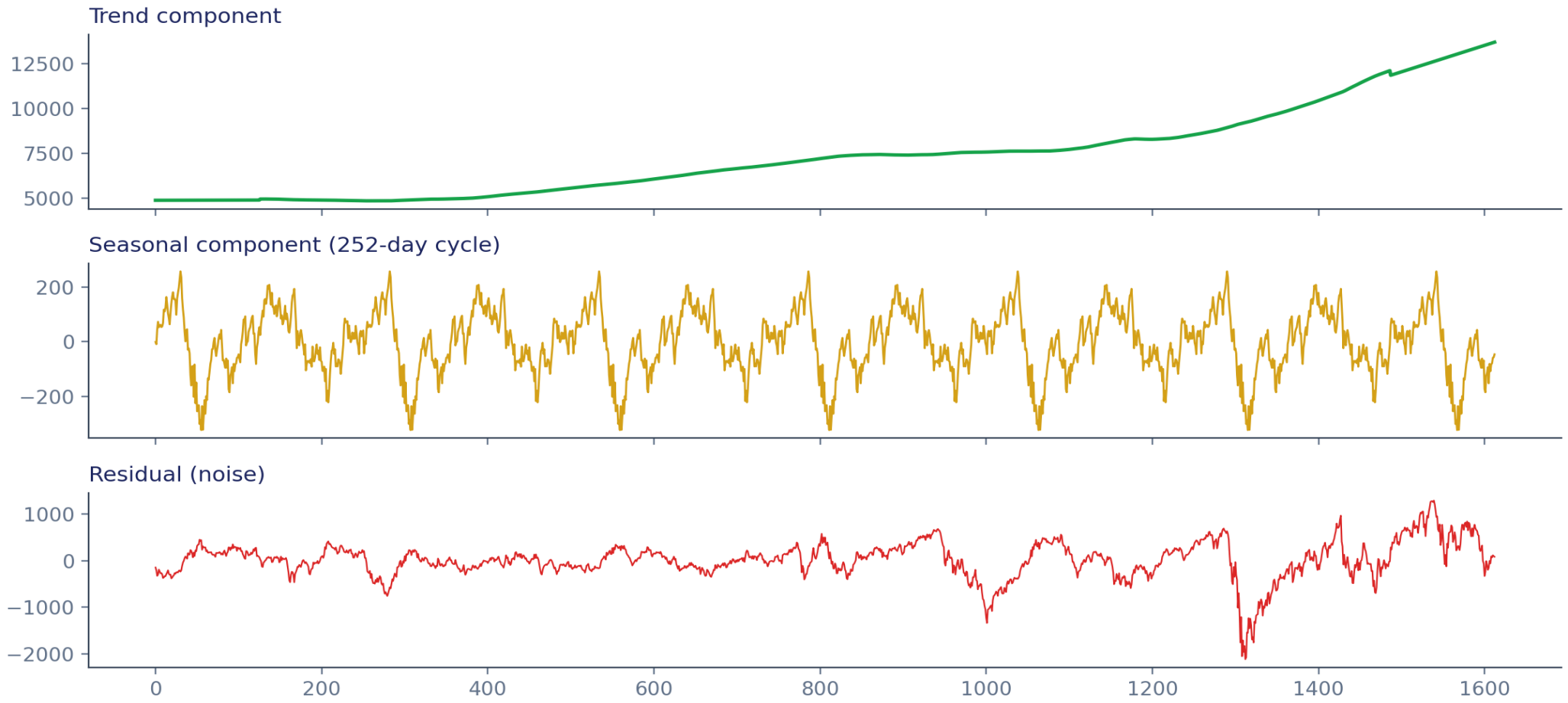
1,613

Span

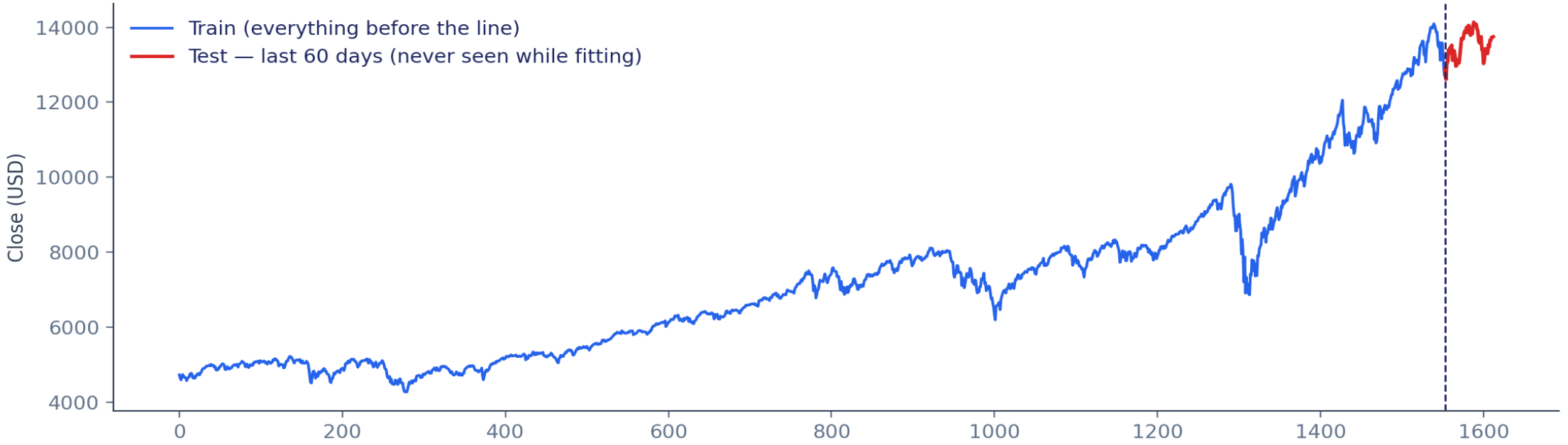
2015 → 2021

Decomposition: Trend + Seasonality + Residual

Every time series can be split into three ingredients. Daily stock prices have almost no calendar seasonality — but the diagnostic is essential for any new series.



Never Shuffle. Never Peek at the Future.



Every model in this lecture — classical, ML, and deep learning — trains only on data strictly **before** the last 60 trading days (~3 months), and is scored only on that untouched window. Same cutoff, same test set, one honest scoreboard at the end.

How Do We Score a Forecast? — MAE & RMSE

MAE — Mean Absolute Error

$$MAE = (1/n) \cdot \Sigma |actual - predicted|$$

Average size of the error, in the same units as the price (dollars). Every error counts equally — a $50 miss counts the same whether it happened on a calm day or a crazy one.

Use when: all errors matter equally, and you want an easy number to explain to a non-technical audience ('on average, we're off by $X').

RMSE — Root Mean Squared Error

$$RMSE = \sqrt{[(1/n) \cdot \Sigma (actual - predicted)^2]}$$

Squares the errors before averaging, then takes the square root. Big misses get punished disproportionately — one terrible day hurts RMSE far more than MAE.

Use when: large errors are especially costly (e.g. risk management) — RMSE ≥ MAE always, and the gap between them tells you how 'spiky' your errors are.

MAPE, and Picking the Right Metric

MAPE — Mean Absolute Percentage Error

$$MAPE = (1/n) \cdot \Sigma \, |actual - predicted| \, / \, |actual| \times 100\%$$

Expresses error as a percentage of the actual value — scale-free, so you can compare forecast quality across a $150 stock and a $15,000 index on equal footing. A MAPE of 6% means predictions are, on average, 6% off the true price.

⚠ *Watch out: MAPE explodes when the actual value is near zero, and it penalizes over-forecasts more than under-forecasts — it isn't symmetric.*

Today's scoreboard reports all three side by side — MAE, RMSE and MAPE — so no single metric's blind spot can hide a bad model.

Quick Picking Guide

Explaining to stakeholders	**MAE**
Penalizing big misses	**RMSE**
Comparing across assets/scales	**MAPE**
Presence of near-zero values	**avoid MAPE**

1. MAE — Mean Absolute Error

Best for: Explaining the average error in an easy way.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

Example:

Actual prices: $100, $110, $120

Predicted: $102, $106, $125

Errors:

- $|100 - 102| = 2$
- $|110 - 106| = 4$
- $|120 - 125| = 5$

$$MAE = \frac{2 + 4 + 5}{3} = 3.67$$

👉 **On average, the prediction is off by $3.67.**

2. RMSE — Root Mean Squared Error

Best for: Penalizing **big mistakes** more heavily.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Using the same errors:

$$RMSE = \sqrt{\frac{2^2 + 4^2 + 5^2}{3}}$$

$$= \sqrt{\frac{4 + 16 + 25}{3}} = \sqrt{15} \ = 3.87$$

👉 A large prediction error gets **squared**, so RMSE punishes big misses.

3. MAPE — Mean Absolute Percentage Error

Best for: Comparing errors across different stock prices/scales.

$$MAPE = \frac{100}{n} \sum_{i=1}^{n} \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Example:

$$MAPE = \frac{100}{3} \left(\frac{2}{100} + \frac{4}{110} + \frac{5}{120} \right)$$

$$= 3.41\%$$

👉 The predictions are approximately **3.41% wrong on average**.

4. Avoid MAPE — When actual values are near zero

MAPE has a problem because we **divide by the actual value**.

For example:

Actual = $0.10
Predicted = $0.20

$$\left| \frac{0.10 - 0.20}{0.10} \right| \times 100 = 100\%$$

Even a $0.10 error becomes a **100% error**.

If actual price is **$0**:

$$\frac{|0 - \hat{y}|}{0}$$

❌ **Cannot divide by zero.**

👉 So when your data contains **zero or near-zero values, avoid MAPE** and consider **MAE or RMSE** instead.

Easy memory trick

Metric	Think of it as	Stock example
MAE	Average dollar error	“Off by $3.67”
RMSE	Punishes big errors	“Big misses hurt more”
MAPE	Average % error	“Off by 3.41%”
Avoid MAPE	Zero/near-zero actuals	“% error can explode”

Super simple:

MAE = dollars wrong → RMSE = big errors matter → MAPE = percentage wrong → near zero = don't use MAPE.

Stationarity & the ADF Test

Most classical models assume a stationary series — constant mean, constant variance, no trend. Raw stock prices trend, so we need a test.

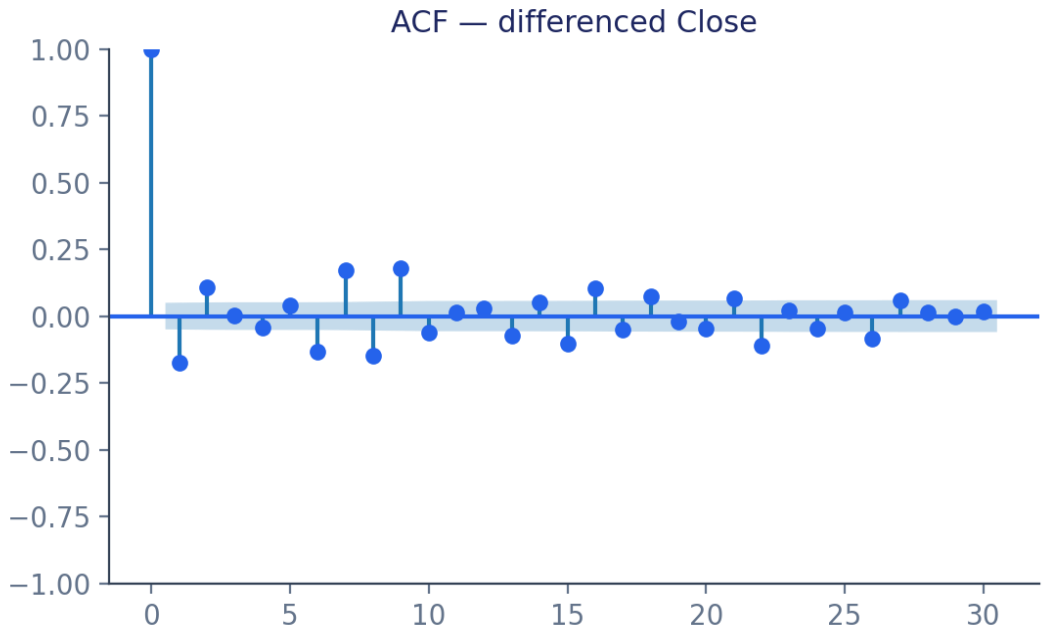
Augmented Dickey-Fuller (ADF) Test

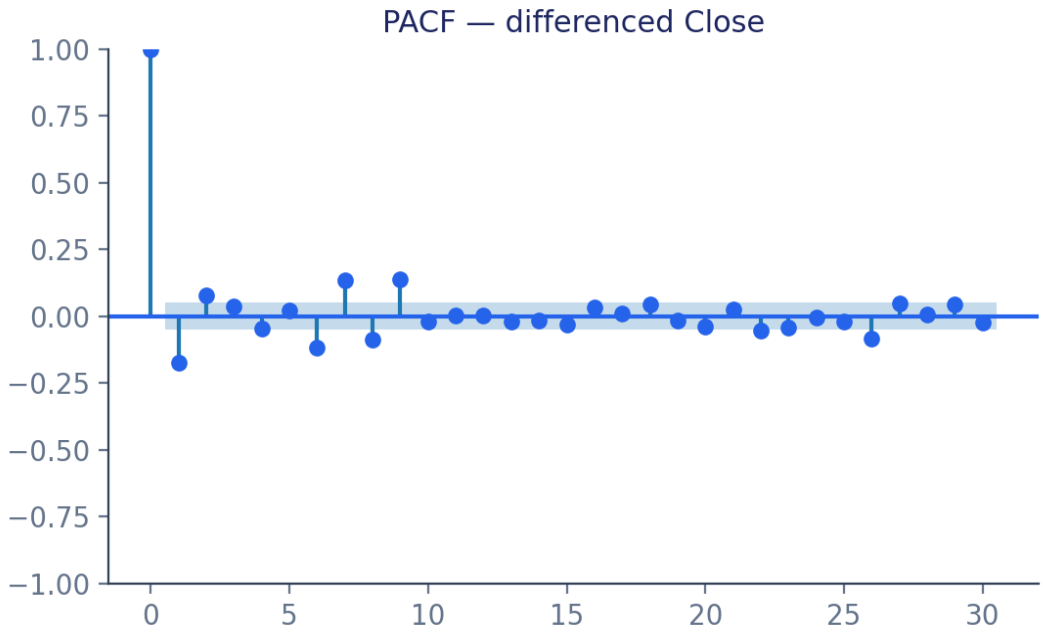
H_0 (null hypothesis): the series is non-stationary. If p-value < 0.05 → reject H_0 → the series is (likely) stationary.

Raw Close price	ADF stat ≈ −1.2 | p-value ≈ 0.66	❌ NON-stationary
1st difference (xt − xt−1)	ADF stat ≈ −11.4 | p-value ≈ 0.00	✅ Stationary

This is exactly the "I" (Integrated) in ARIMA — differencing once removes the trend and makes the series usable by AR/MA machinery.

ACF & PACF: Choosing the Model Order





ACF (Autocorrelation) — correlation with lagged values. Tails off slowly → points to the MA order (q).

PACF (Partial Autocorrelation) — correlation after removing shorter lags. Sharp cutoff at lag p → points to the AR order (p).

The Classical Family, Building Block by Block

Model	Captures	statsmodels class
Naive	Tomorrow = today	`the baseline every model must beat`
AR(p)	Value depends on its own past p values	`ARIMA(order=(p,0,0))`
MA(q)	Value depends on past q forecast errors	`ARIMA(order=(0,0,q))`
ARMA(p,q)	Both AR and MA combined	`ARIMA(order=(p,0,q))`
ARIMA(p,d,q)	ARMA + differencing for trend	`ARIMA(order=(p,d,q))`
SARIMA	ARIMA + a seasonal cycle	`SARIMAX(order, seasonal_order)`
Holt-Winters	Trend + seasonality via exponential smoothing	`ExponentialSmoothing(...)`
VAR	Several series forecast jointly	`VAR(...) — multivariate`

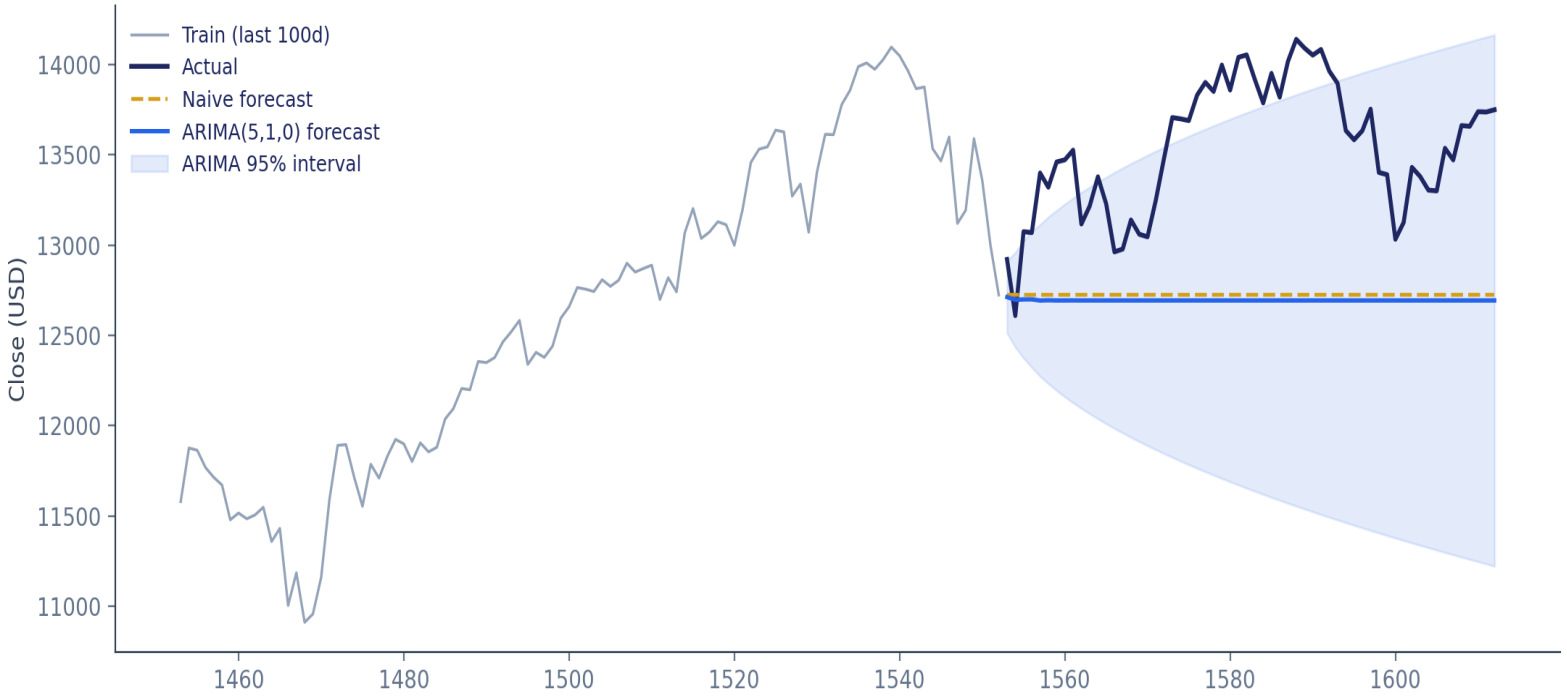
Baseline First: The Naive Forecast

The simplest possible forecast: **“tomorrow = today.”** For a near-random-walk series like a stock index, this is a shockingly strong baseline — if a fancy model can't beat it, the complexity isn't earning its keep.

$822	$895	6.0%
MAE on the 60-day test window	RMSE on the 60-day test window	MAPE on the 60-day test window

This is the exact number every classical, ML and deep learning model in this lecture has to beat.

ARIMA in Action — Point Forecast + Uncertainty



Why the cone matters

A forecast interval widens the further out you predict — uncertainty compounds. A model reporting only a point forecast is quietly hiding how unsure it really is.

Residual diagnostics

For a well-fit model, residuals should look like white noise — no leftover pattern. The Ljung-Box test checks this: p > 0.05 is good.

Real result on IXIC: ARIMA(5,1,0) — RMSE $922, right in line with the naive baseline. On daily prices, that's expected — and instructive.

Exponential Smoothing & Multivariate Forecasting

Exponential Smoothing Ladder

1. **SES**

   Weighted average of the past — weights decay exponentially. Good for a level with no trend.

2. **Holt's Method**

   SES + a trend component — tracks a rising or falling series.

3. **Holt-Winters**

   Holt + a seasonal component — the full package.

VAR — Vector Autoregression

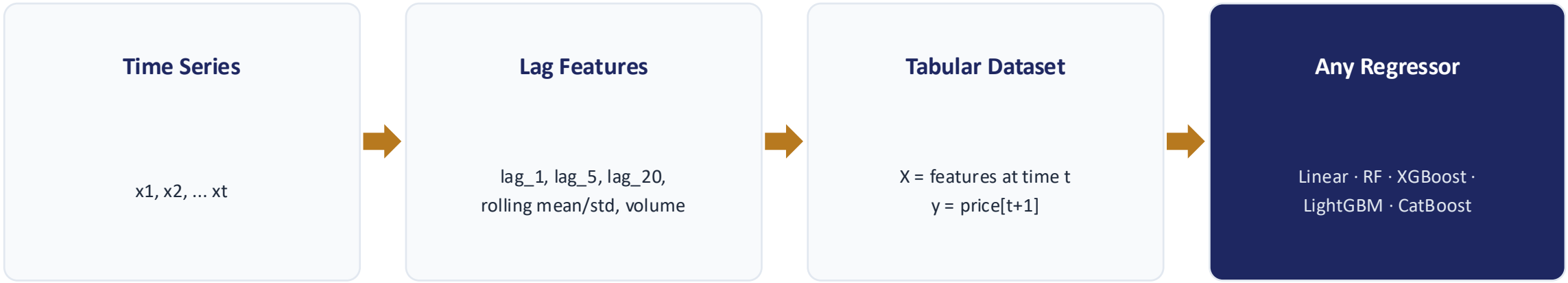
Every model so far is univariate — one series, forecast from its own past. VAR forecasts several series jointly, where each depends on lagged values of all of them.

Example: NASDAQ (IXIC) and NYSE Composite (NYA) are well-correlated US indices — a natural pair for VAR. Inputs are differenced first, since VAR needs stationary series.

Use VAR when your series genuinely move together — interest rates & bond yields, GDP & unemployment, or two correlated markets.

Reshaping a Time Series into a Table

Classical models have a fixed structure (AR order, MA order…). The ML approach turns the problem into ordinary supervised learning.



Crucial detail: every feature at row t must only use information available up to time t. Leak tomorrow's price into today's features and your backtest lies to you.

Tabular Regressors: From Linear to Gradient Boosting

Linear Regression

The simplest baseline regressor over lag features

Random Forest

Bagged decision trees — robust, low tuning effort

XGBoost

Gradient boosting — sequential trees fixing prior errors

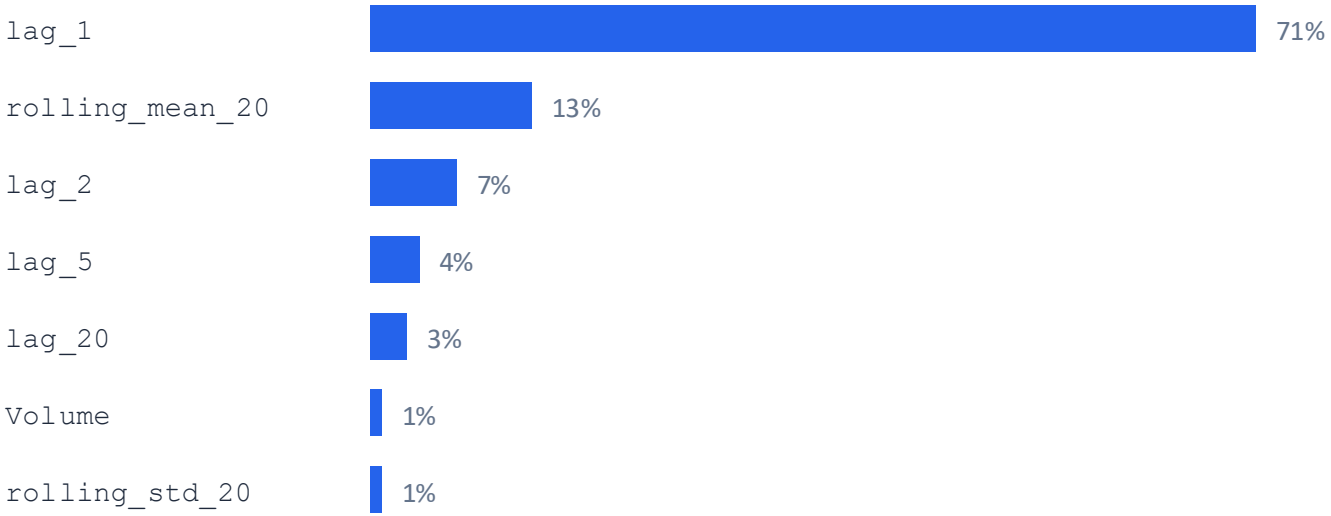
LightGBM

Histogram-based boosting — very fast on large tables

CatBoost

Boosting with strong defaults, great on messy/categorical data

Which feature matters most? (Random Forest importances, real run)



Yesterday's price dominates — another sign the market is close to efficient: hard to beat with more complexity alone.

The Deep Learning Ladder

Instead of hand-crafted lag features, sequence models take a raw sliding window of the last 20 values and learn the pattern themselves.

MLP

1

Baseline

Sees the whole window as one flat vector — no real notion of order.

RNN

2

Recurrence

Processes the window one step at a time, carrying a hidden state — the first model to respect sequence order.

LSTM

3

Long memory

Adds gated memory cells so the network can choose what to remember or forget over long windows.

GRU

4

Efficient

Merges LSTM's gates into fewer parameters — nearly as accurate, faster to train.

1D CNN

5

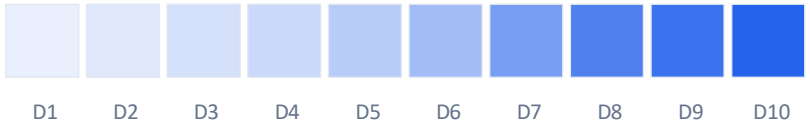
Convolution

Slides learned filters across the window, stacking layers to grow receptive field — same idea behind TCNs used in production.

Attention & Transformers — the Modern Era

RNN/LSTM/GRU process the window step by step — slow, and long-range dependencies must survive many steps. Attention lets every position look directly at every other position, in one shot.

Self-Attention — Day 10 looking back at Days 1-10



Darker = higher attention weight. A causal mask keeps Day 10 from ever attending to the future.

Core Ideas, in Forecasting Order

Attention: For each query position, compute a weighted sum over all other positions — weights reflect relevance.

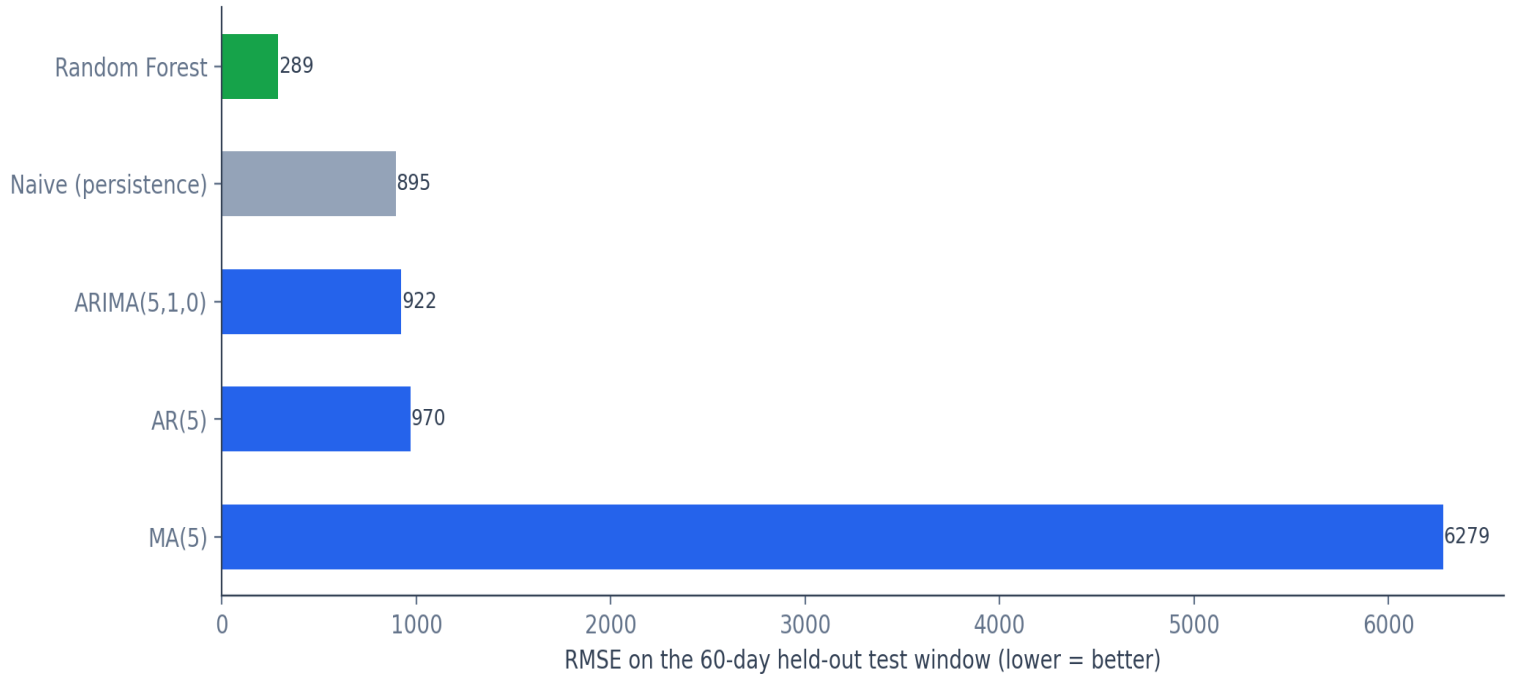
Self-Attention: Query, key & value all come from the same sequence — the model learns which past days matter.

Positional Encoding: Since attention has no built-in order, position info is injected explicitly.

Transformer: Stacks many self-attention blocks + feed-forward layers — the architecture behind modern LLM-style forecasters.

Final Scoreboard — Every Model, Same Test Set

All models scored on the exact same 60-day held-out window of IXIC Close prices. Lower RMSE is better. (Real results from this dataset.)



Reading the board

Random Forest wins here — lag features + tree ensembles picked up short-term structure naive/ARIMA couldn't.

MA(5) collapses toward the historical mean — proof that MA alone is the wrong tool for a trending series.

Naive still beats plain AR/ARIMA — a genuine reminder of how close daily prices are to a random walk.

⚠ *Caveat: classical models forecast the whole 60-day window blind from one cutoff — a harder task than ML's step-by-step lag features. Compare families thoughtfully, not just by rank.*

How to Read a Forecast Plot Like a Pro

1

Check it against the naive baseline first.

If a complex model can't beat "tomorrow = today," its complexity isn't earning its keep — especially true for near-random-walk daily prices.

2

Look at the interval, not just the line.

A point forecast with no uncertainty band is overclaiming confidence. Prefer models (ARIMA, SARIMA) that report one.

3

Check the residuals, not just the fit.

A model can track the training data closely and still leave structured, non-random errors behind — that structure is a warning sign.

RECAP

What You Just Learned — Level by Level

1	**Foundations**	Trend, seasonality, stationarity, the chronological train/test split, MAE / RMSE / MAPE
2	**Classical**	Naive, AR, MA, ARMA, ARIMA, SARIMA, SES, Holt, Holt-Winters, VAR — explicit statistical structure
3	**Tabular ML**	Linear Regression, Random Forest, XGBoost, LightGBM, CatBoost — reshape the series into lag features
4	**Deep Learning**	MLP → RNN → LSTM → GRU → 1D CNN — learn the pattern directly from a raw sliding window
5	**Modern Era**	Self-attention & Transformers — every position attends to every other, in one shot

Thank You

Questions? Let's talk models, metrics, and markets.

CP020003 · Artificial Intelligence 2026